

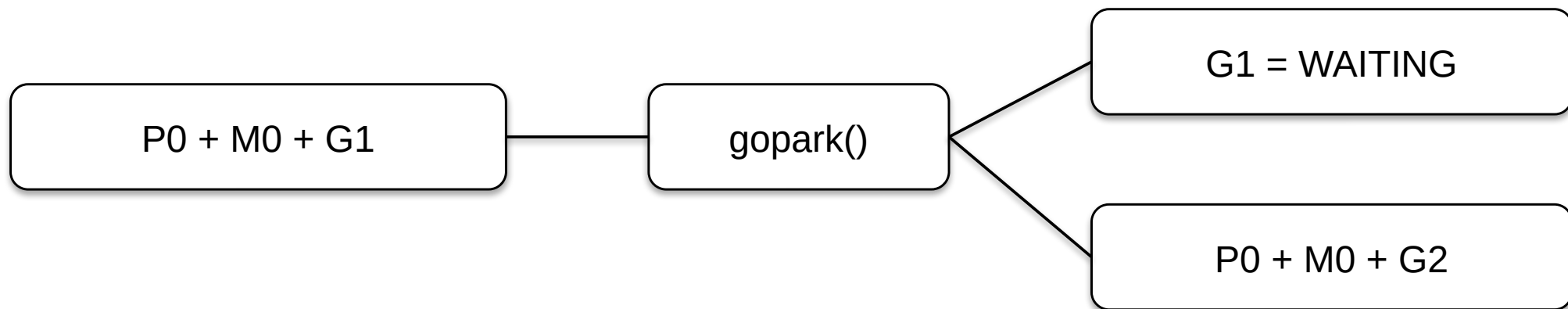
Go runtime synchronization internals

gopark / goready / sudog / semaphore / sync.Mutex

Главный вопрос

- Что происходит, когда `<-ch` или `mu.Lock()` прямо сейчас не могут продолжить выполнение?
- Почему ожидающая goroutine обычно не удерживает заблокированный OS thread?
- Как runtime запоминает waiter и как возвращает его обратно scheduler?

Блокируем G, а не M

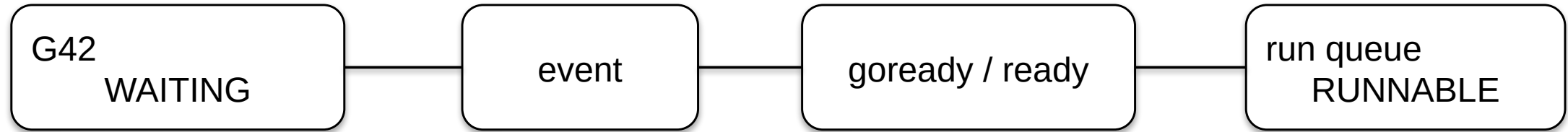


Ключ: ожидание scheduler-aware primitive паркует G; M и P остаются полезными для других runnable goroutines.

gorock: RUNNING → WAITING

- gorock означает: текущая goroutine сейчас не может продолжать работу.
- Runtime переключается на system stack g0, переводит G в _Gwaiting и отвязывает её от M.
- Текущий M возвращается в schedule() и получает другую runnable G.

goready: WAITING → RUNNABLE

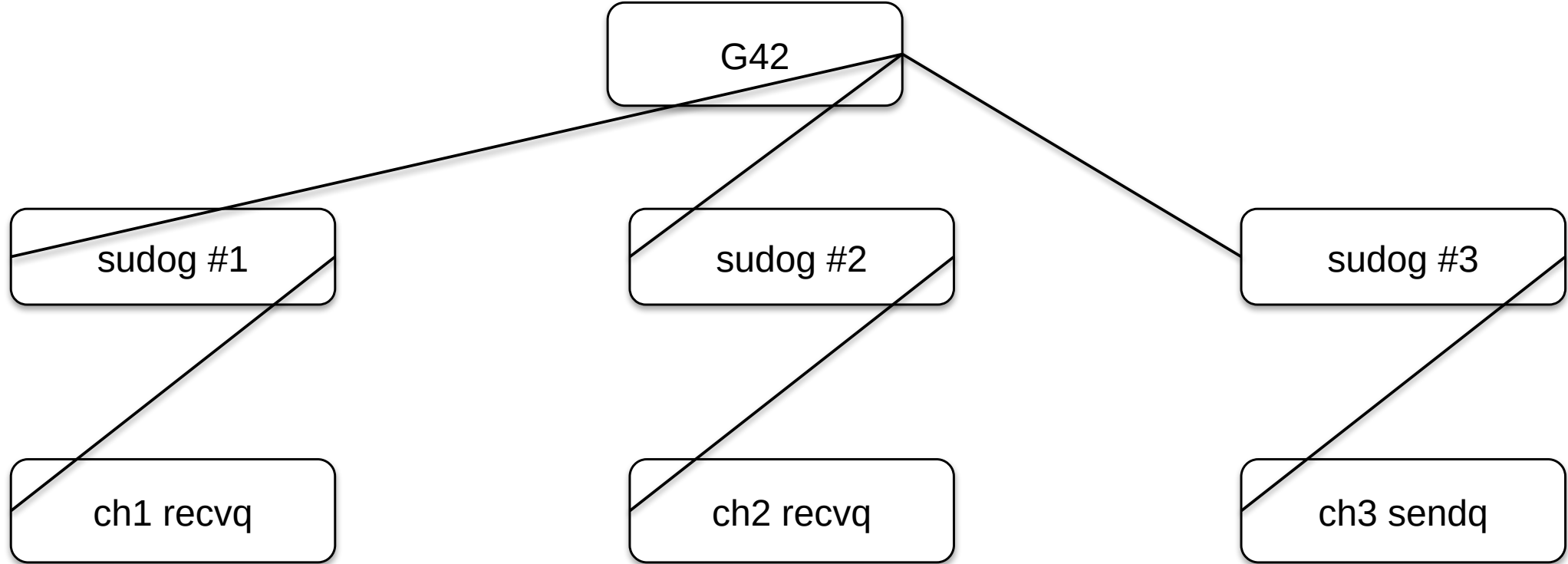


goready не означает «исполняй сейчас». Она только возвращает G в конкуренцию за P/M.

Что такое `sudog`

- `sudog` — runtime-запись `goroutine` в очереди ожидания конкретного `synchronization object`.
- Она связывает `G` с объектом ожидания: `channel`, `semaphore` и другими `wait structures`.
- `sudog` — не сама `goroutine`; это отдельный `waiter node`.

Почему не хранить просто *g

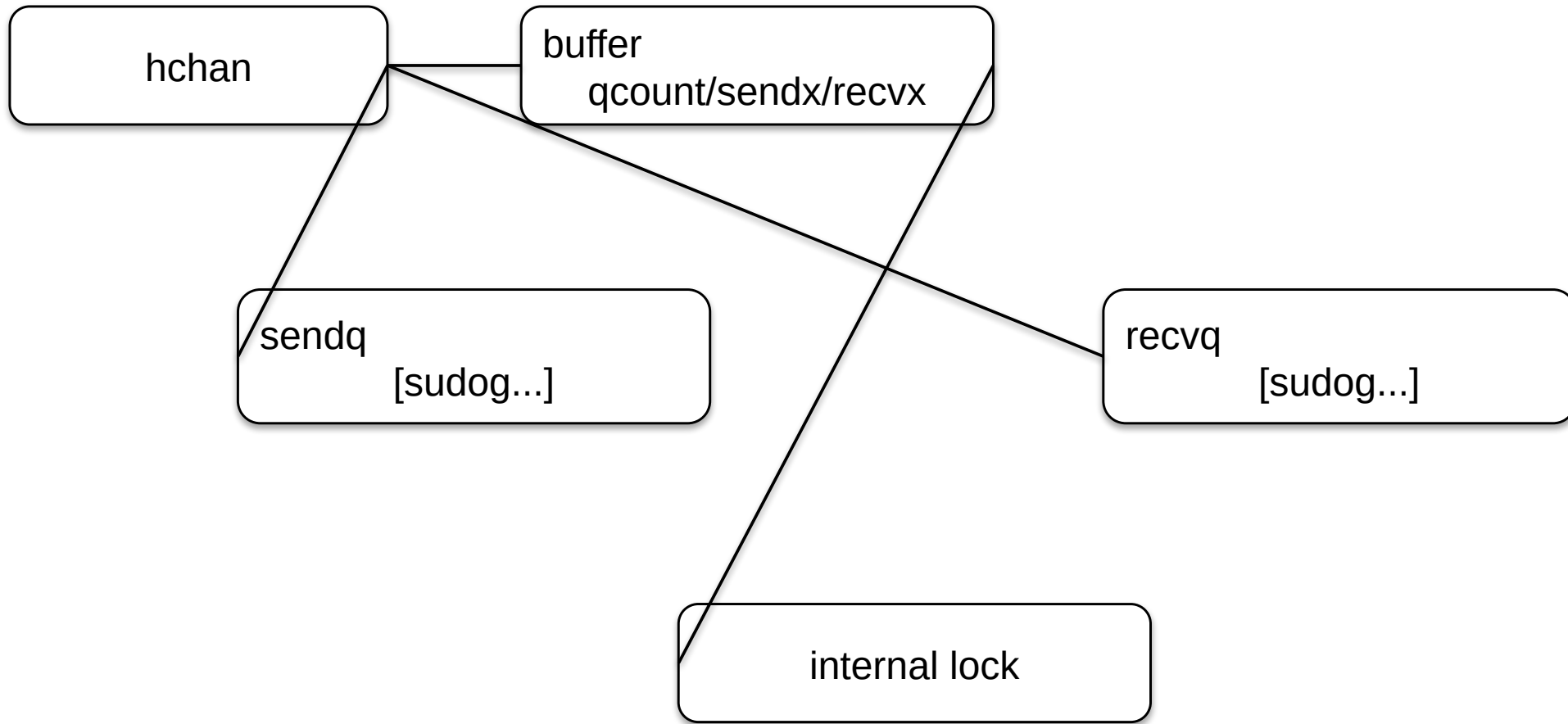


select может зарегистрировать одну G сразу в нескольких wait queues → нужна many-to-many связь через sudog.

sudog pool

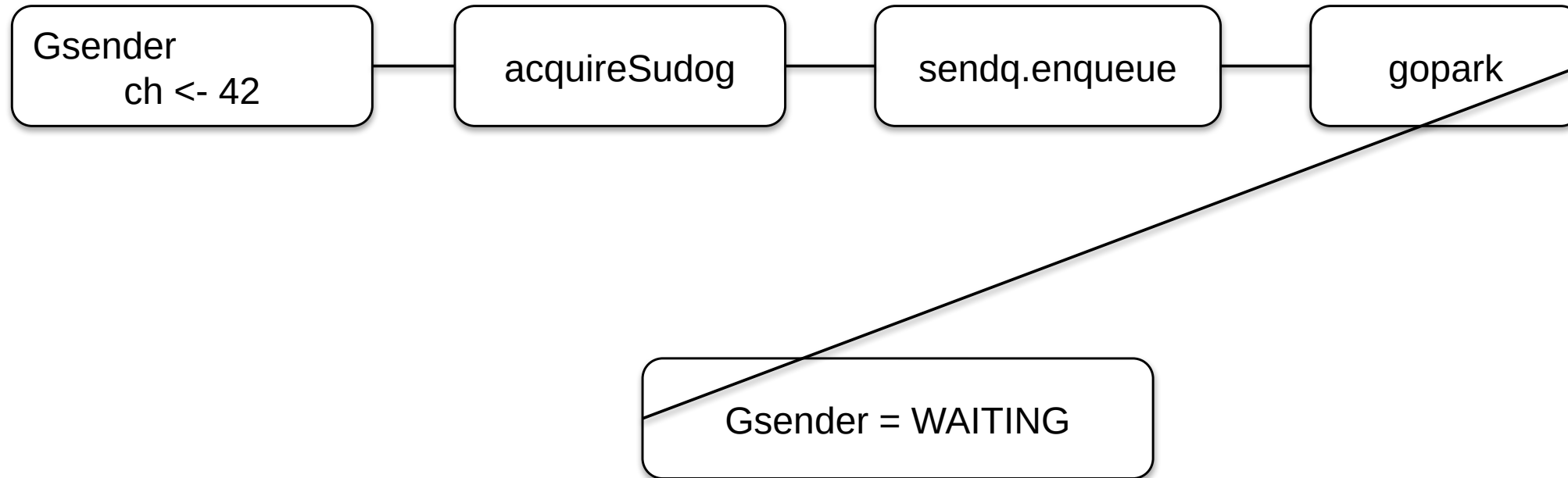
- Blocking operations очень частые; malloc/free sudog на каждое ожидание был бы дорогим.
- Runtime использует acquireSudog()/releaseSudog() и переиспользует waiter nodes.
- Это типичный Go runtime-подход: дешёвый fast path + специальные пулы для горячих внутренних объектов.

Channel: внутренние очереди ожидания



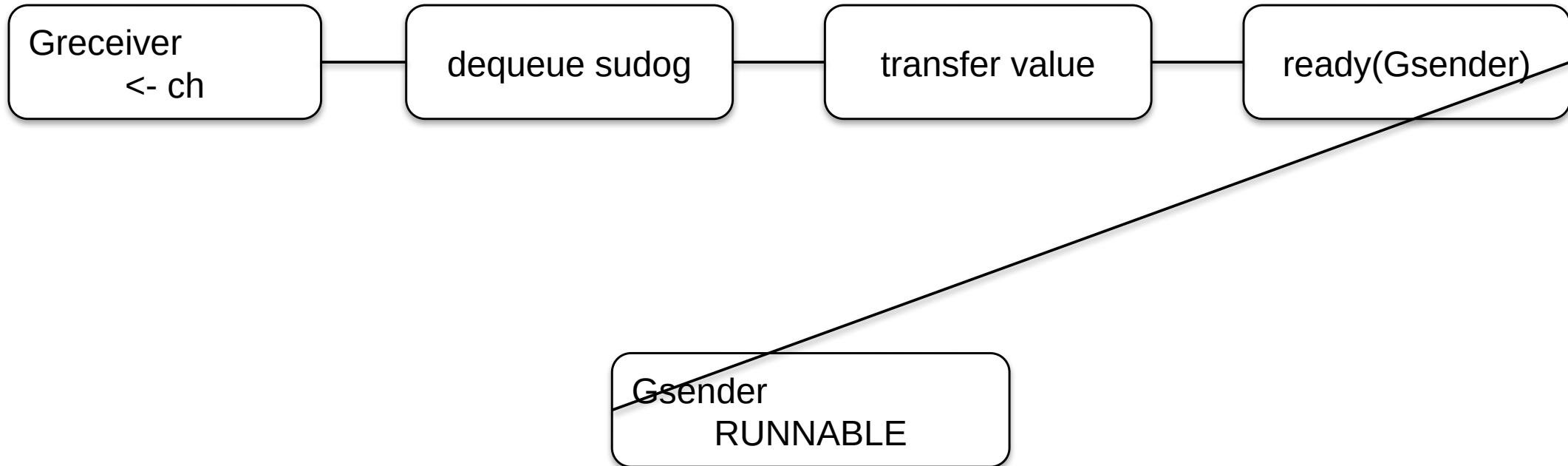
sendq хранит заблокированных sender, recvq — receiver. Lock защищает state machine channel, а не пользовательское ожидание целиком.

Unbuffered send: receiver отсутствует



sudog хранит ссылку на G и данные/адрес элемента, необходимые для rendezvous.

Receiver приходит позже



Пробуждённый sender снова идёт через scheduler; receiver не «вставляет» его прямо на CPU.

Lost wakeup: главный concurrency-риск

- Плохой алгоритм: check condition → context switch → signal без waiter → park.
- Если waiter ещё не зарегистрирован, сигнал может потеряться, а G заснуть навсегда.
- Поэтому register waiter, release internal lock и park должны образовывать согласованный protocol.

Зачем gorpark получает unlock callback

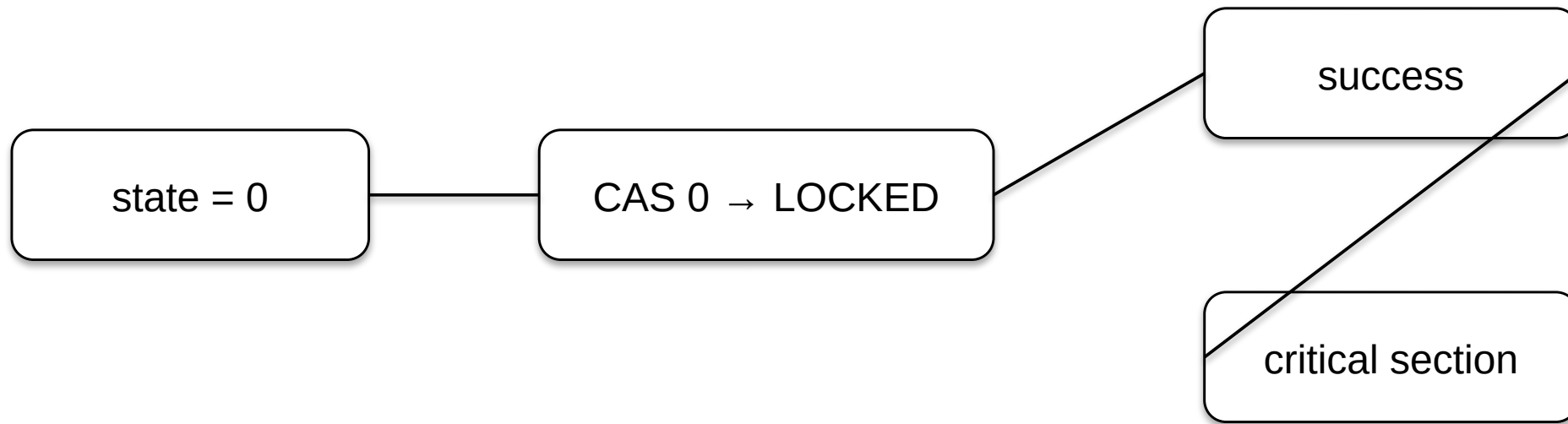
- Runtime должен выпустить internal lock в правильный момент — после регистрации waiter, но до сна.
- Так signal-side либо видит ещё незапаркованную G под lock, либо уже видит её в wait queue.
- Это защита от missed wakeup; тот же класс проблемы есть у condition variables/futex/semaphore.

sync.Mutex: маленькая структура, сложный protocol

```
type Mutex struct {  
    state int32  
    sema  uint32  
}
```

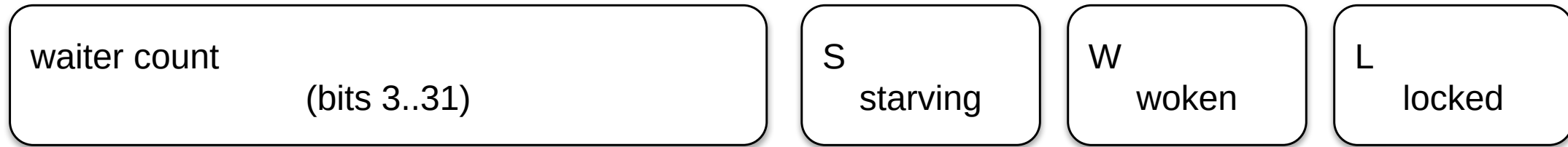
```
    // state: locked / woken / starving / waiter count
```

Mutex fast path: CAS



Uncontended mutex: atomics и return. Никаких sudog, semaphore, park и scheduler slow path.

Mutex state: packed bits

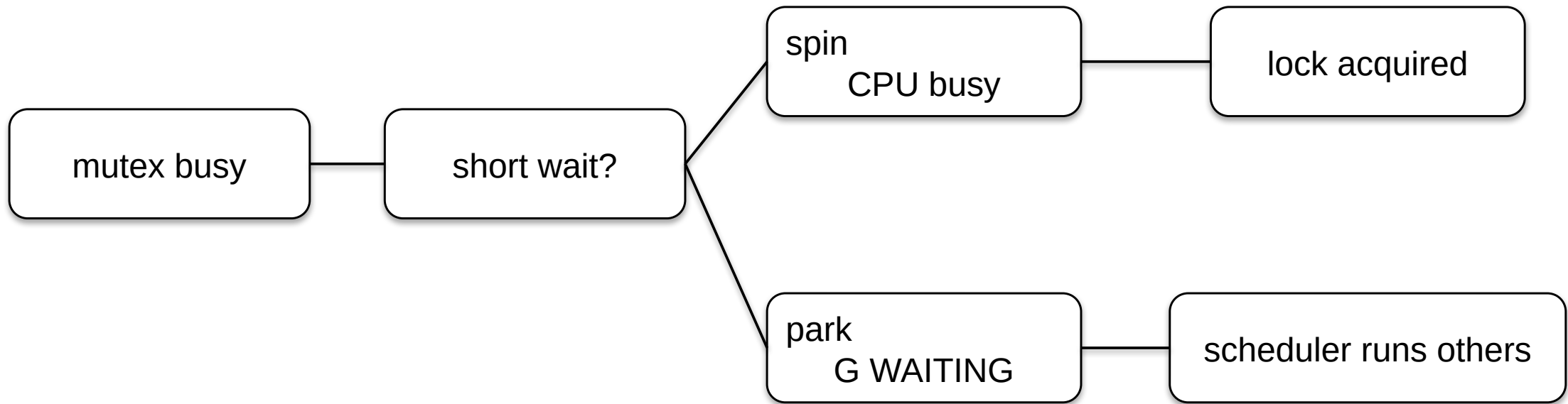


Один int32 хранит lock state, признак уже пробуждённого waiter, starvation mode и число ожидающих.

Lock slow path: зачем spinning

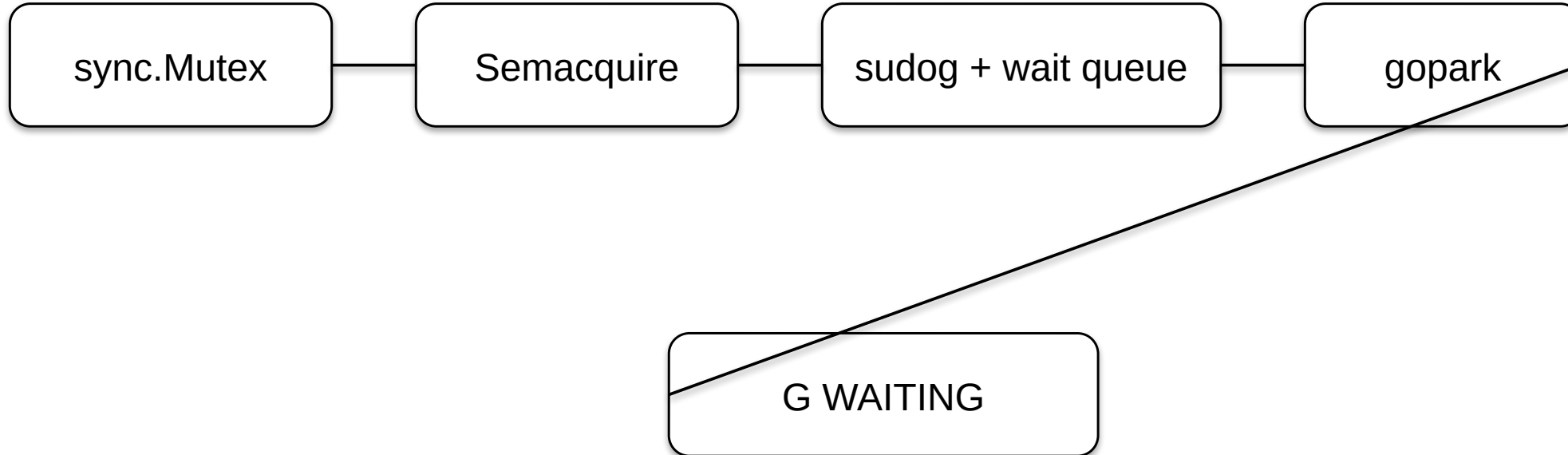
- CAS проигран: mutex занят. Но critical section может закончиться быстрее, чем стоит park/unpark.
- Runtime разрешает короткое active spinning: G остаётся RUNNING и проверяет lock ещё несколько раз.
- Spinning = обмен CPU на меньшую latency синхронизации.

Spinning vs parking



Runtime spin'ится консервативно: multicore, свободный смысл в spinning и отсутствие более полезной runnable работы на локальном P.

Runtime semaphore: мост к scheduler

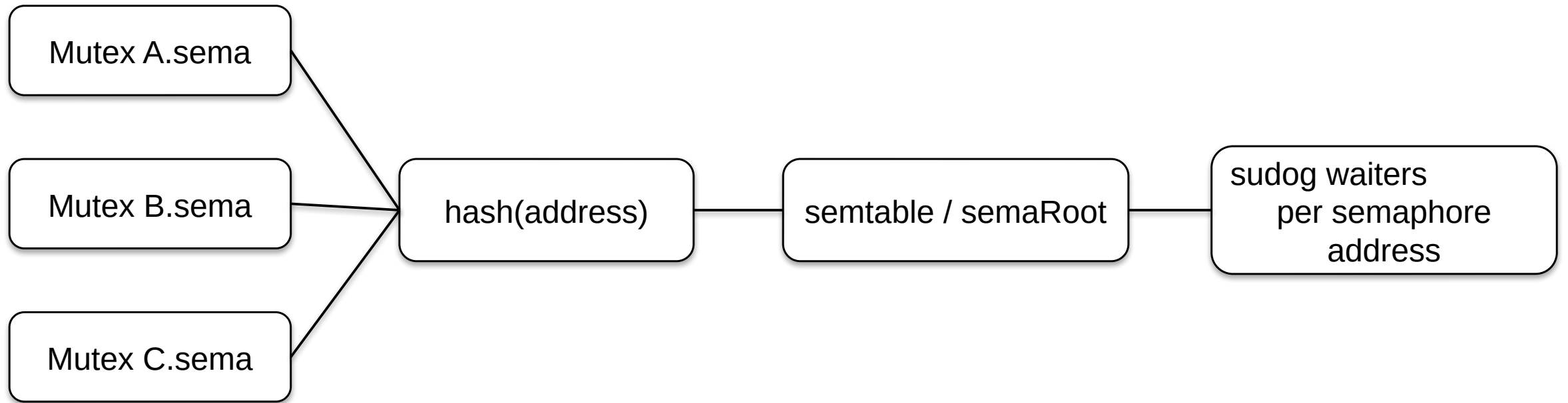


Это runtime semaphore, а не обязательный kernel semaphore: его задача — асинхронно park/ready goroutines.

semacquire1: почему condition проверяется повторно

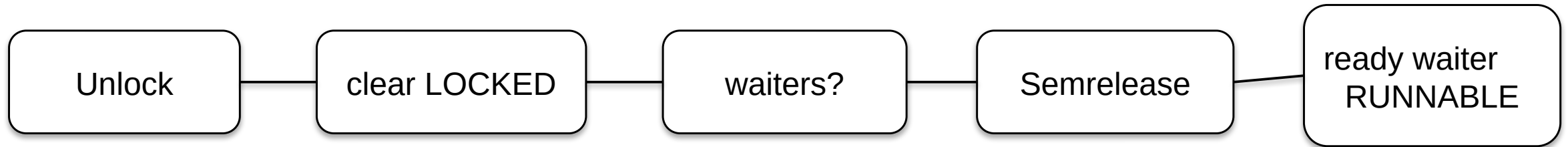
- Fast path пытается забрать semaphore token атомарно.
- Если token нет: увеличиваем waiter count, затем перепроверяем token перед enqueue/park.
- Повторная проверка закрывает окно гонки между первой проверкой и регистрацией waiter.

Где живут Mutex waiters



Очередь не лежит в каждом Mutex: сам mutex остаётся маленьким, тяжёлая wait-инфраструктура общая и включается только при contention.

Unlock → Semrelease → ready

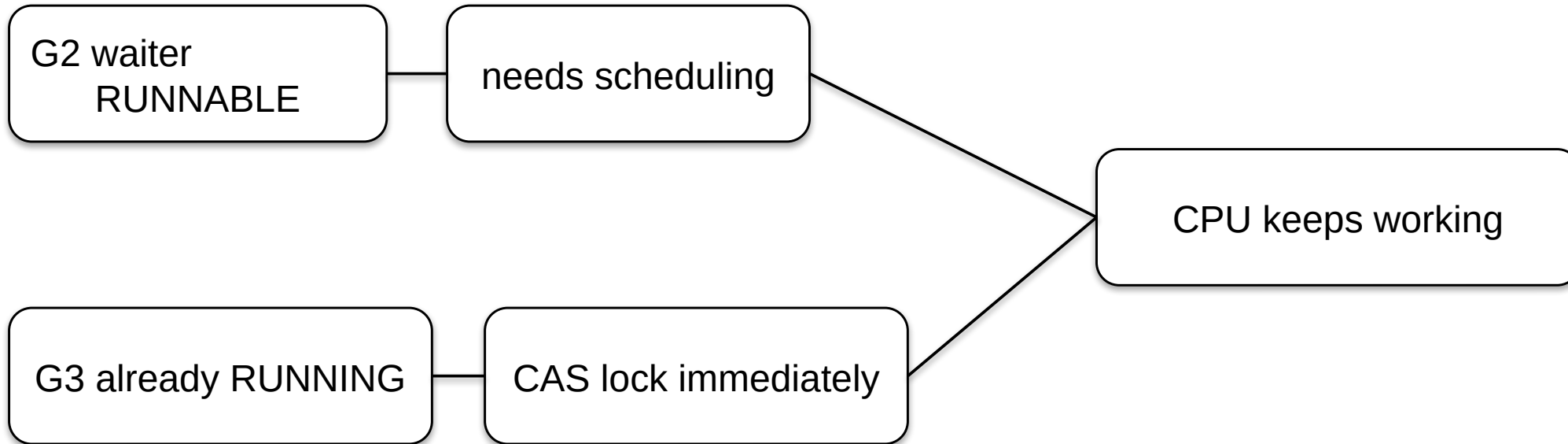


Fast path unlock может завершиться одной atomic операцией; scheduler подключается только если есть waiter/state для slow path.

Normal mode: barging

- Пробуждённый waiter ещё должен получить CPU и снова конкурировать за mutex.
- Новая goroutine уже RUNNING и может успеть захватить освободившийся lock раньше старого waiter.
- Это barging: throughput выше, но старый waiter потенциально проигрывает несколько раз подряд.

Почему barging выгоден throughput



Normal mode предпочитает already-running contender → меньше handoff overhead и выше throughput.

Starvation mode

- Если waiter слишком долго проигрывает, текущая реализация может переключить mutex в starvation mode.
- В starvation mode ownership фактически передаётся следующему waiter; newcomers не должны steal lock.
- Trade-off: fairness и p99 лучше, throughput хуже. Порог порядка 1 ms — implementation detail, не API contract.

Normal vs starvation

NORMAL

Unlock

wake waiter

newcomer may
steal

STARVATION

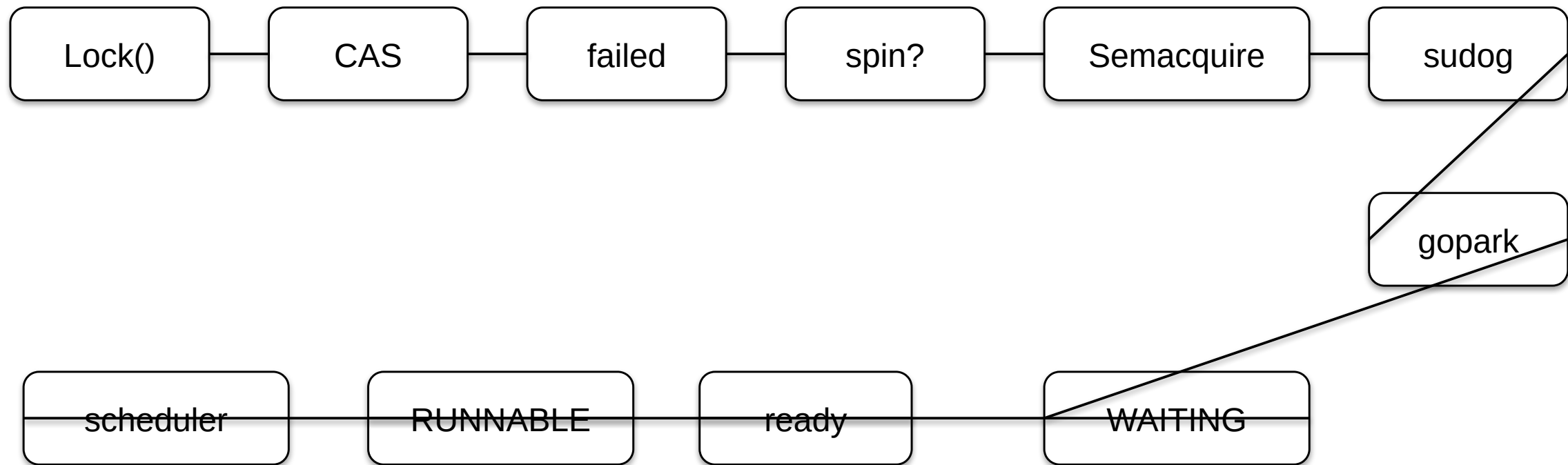
Unlock

direct handoff

newcomer queues

Normal optimizes throughput. Starvation mode optimizes fairness/tail latency.

Полный Mutex.Lock slow path



Fast path заканчивается на CAS. Весь остальной runtime-механизм появляется только при contention.

Channel и Mutex: один runtime-паттерн

- Channel: wait queues sendq/recvq находятся внутри hchan.
- Mutex: waiters обслуживаются runtime semaphore table.
- Общая идея одинакова: register waiter → sudog → gopark → event → ready/goready → RUNNABLE.

Что происходит под contention

- Много goroutines могут быть WAITING на одном mutex при низкой загрузке CPU.
- Это serialization bottleneck, а не scheduler saturation.
- Рост contention добавляет spinning, park/ready, queueing и scheduler latency вокруг самой критической секции.

Backend-пример: глобальный cache mutex

- На TEST 10 RPS: Lock → map lookup → Unlock выглядит бесплатным.
- На PROD 50 000 RPS: тысячи G конкурируют за одну critical section.
- Симптомы: p99 растёт, block/mutex profile показывает hotspot, CPU может быть далёк от 100%.

Диагностика

- Mutex profile: где goroutines теряют время в contested locks.
- Block profile: channel/mutex/Cond/select blocking.
- go tool trace: RUNNING → WAITING(sync.Mutex.Lock) → RUNNABLE → RUNNING.
- runtime.NumGoroutine сам по себе недостаточен: нужен wait reason и время ожидания.

Практика

- Benchmark uncontended Mutex: доказать дешевизну CAS fast path.
- RunParallel + разная длительность critical section: увидеть переход spin → park.
- Unbuffered channel: через trace увидеть sendq/recvq park/ready.
- 100 000 goroutines ждут channel: сравнить goroutine count и OS thread count.

Что важно запомнить

- gopark паркует G; goready/ready возвращает её в RUNNABLE.
- sudog — waiter node, связывающий G с synchronization object.
- Mutex fast path — atomics; slow path — spinning → semaphore → parking.
- Semaphore wait queues позволяют заблокировать goroutine без отдельного blocked OS thread.
- Normal/starvation modes балансируют throughput и fairness.
- Под нагрузкой проблема часто не в mutex как инструкции, а в очереди перед critical section.